

Modification Plan: quadrature_analyzer.py API Alignment

Objective

Modify `api/quadrature_analyzer.py` so that its internal integration helpers (`_integrate_*`) use the **correct, existing public APIs** of the four orthogonal polynomial modules. The existing module code in `chebyshev/`, `hermite/`, `laguerre/`, and `legendre/` directories must **never be modified**.

Existing Module API Reference (Read-Only)

Legendre (`legendre/integration.py`)

Symbol	Signature	Returns
<code>'LegendreQuadrature(n, use_mpmath=False, dps=80)'</code>	Constructor	Instance with <code>'nodes'</code> , <code>'weights'</code> properties
<code>'LegendreQuadrature.integrate(f)'</code>	Method	<code>'float'</code> — integral over canonical <code>[-1, 1]</code>
<code>'GaussLegendreQuadrature.gauss_levsch(n)'</code>	Static method	<code>'(np.ndarray nodes, np.ndarray weights)'</code>

Chebyshev (`chebyshev/integration.py`)

Symbol	Signature	Returns
<code>'clencurt_quadrature(f, n)'</code>	Function	<code>'float'</code> — Clenshaw-Curtis integral over <code>[-1, 1]</code>
<code>'clencurt_integrate_interval(f, a, b, n=32)'</code>	Function	<code>'float'</code> — Clenshaw-Curtis integral over <code>[a, b]</code> with correct Jacobian scaling
<code>'ChebyshevQuadrature.clenshaw_curtis_quadrature(f, n)'</code>	Method	<code>'float'</code> — same as <code>'clencurt_quadrature'</code>
<code>'ChebyshevQuadrature.integrate(f, a, b, n=32)'</code>	Method	<code>'float'</code> — same as <code>'clencurt_integrate_interval'</code>

Hermite (`hermite/integration.py`)

Symbol	Signature	Returns
<code>'GaussHermiteQuadrature(n, use_mpmath=False, dps=80)'</code>	Constructor	Instance with <code>'integrate(f)'</code> method
<code>'GaussHermiteQuadrature.integrate(f)'</code>	Method	<code>'float'</code> — integral over <code>(-inf, +inf)</code> with e^{-x^2} weight

Laguerre (`laguerre/integration.py`)

Symbol	Signature	Returns
<code>'LaguerreQuadrature(n, alpha=0.0, use_mpmath=False)'</code>	Constructor	Instance with <code>'nodes'</code> , <code>'weights'</code> , <code>'integrate(f)'</code>
<code>'LaguerreQuadrature.integrate(f)'</code>	Method	<code>'float'</code> — integral over <code>[0, +inf)</code> with e^{-x} weight

'gauss_quadrature_weights(n,Function alpha=0.0)'	'np.ndarray weights' only (NOT nodes+weights tuple)
---	--

Issues Found in quadrature_analyzer.py

ISSUE 1: `_integrate_legendre` — calls non-existent `integrate_transformed()`

Location: Lines 981-991, and usage snippet at lines 794-799.

```
# CURRENT (BROKEN):
quad = LegendreQuadrature(n=n, use_mpmath=False)
result = quad.integrate_transformed(func, a=a, b=b, nodes=quad.nodes,
    ↪ weights=quad.weights)
```

Problem: `LegendreQuadrature` has NO method called `integrate_transformed`. It only has `.integrate(f)` which integrates over the canonical `[-1, 1]` interval.

Fix: Manually transform from `[a, b]` to `[-1, 1]` and use `.integrate()`:

```
# FIXED:
scale = (b - a) / 2.0
shift = (b + a) / 2.0
quad = LegendreQuadrature(n=n, use_mpmath=False)
result = quad.integrate(lambda x: func(scale * x + shift))
return float(result) * scale
```

Also fix the `recommend_usage()` snippet at lines 794-799 to use `.integrate()` with a transformed lambda instead of `.integrate_transformed()`.

ISSUE 2: `_integrate_chebyshev` — calls non-existent `gauss_chebyshev_quadrature()`

Location: Lines 994-1008, and usage snippet at lines 807-809.

```
# CURRENT (BROKEN):
q = ChebyshevQuadrature()
return float(q.gauss_chebyshev_quadrature(stripped, n=n))
```

Problem: `ChebyshevQuadrature` has NO method called `gauss_chebyshev_quadrature`. It has `.clenshaw_curtis_quadrature(f, n)` and `.integrate_on(f, a, b, n)`.

Fix for non-singularity path (lines 1004-1008): Use the existing `clencurt_integrate_interval` function which already handles interval mapping with correct Jacobian:

```
# FIXED (non-singularity):
from chebyshev import clencurt_integrate_interval
return float(clencurt_integrate_interval(func, a, b, n))
```

Fix for singularity path (lines 997-1003): Use `.clenshaw_curtis_quadrature()` instead of the non-existent method:

```
# FIXED (singularity):
from chebyshev import ChebyshevQuadrature
q = ChebyshevQuadrature()
return float(q.clenshaw_curtis_quadrature(stripped, n=n))
```

Also fix the `recommend_usage()` snippet at lines 807-809.

ISSUE 3: `_integrate_laguerre` — uses low-level API instead of class API

Location: Lines 1029-1036, and usage snippet at lines 824-825.

```
# CURRENT (INCONSISTENT):
from laguerre import gauss_quadrature_weights
nodes, weights = gauss_quadrature_weights(n) # WRONG: returns only weights!
return float(np.sum(weights * stripped(nodes)))
```

Problem: `gauss_quadrature_weights(n)` returns ONLY the weights array (not a tuple of nodes+weights). The current code tries to unpack it as `(nodes, weights)` which will fail. Even if it worked, this is inconsistent with how other families use their class-based `.integrate()` method.

Fix: Use `LaguerreQuadrature` class consistently:

```
# FIXED:
from laguerre import LaguerreQuadrature
quad = LaguerreQuadrature(n=n, alpha=0.0, use_mpmath=False)
return float(quad.integrate(stripped))
```

ISSUE 4: `_integrate_hermite` — API is correct but verify consistency

Location: Lines 1010-1027.

```
# CURRENT (CORRECT):
from hermite import GaussHermiteQuadrature
quad = GaussHermiteQuadrature(n=n, use_mpmath=use_mpmath)
return float(quad.integrate(stripped))
```

Status: This is already correct. `GaussHermiteQuadrature` has `.integrate(f)` and the usage matches. No change needed here.

Summary of Required Changes

Method	Current Problem	Fix
<code>'_integrate_legendre()'</code> (line 981-991)	Calls non-existent <code>'integrate_transformed()'</code>	Use <code>'integrate(lambda x: func(scale*x+shift))'</code> with Jacobian scaling
<code>'_integrate_chebyshev()'</code> singularity path (line 1003)	Calls non-existent <code>'gauss_chebyshev_quadrature()'</code>	Use <code>'clen-curt_integrate(f, n)'</code>
<code>'_integrate_chebyshev()'</code> non-singularity path (lines 1004-1008)	Manual mapping + <code>'clen-curt_quadrature'</code>	Use <code>'clen-curt_integrate_interval(func, a, b, n)'</code> directly
<code>'_integrate_laguerre()'</code> (line 1029-1036)	Misuses <code>'gauss_quadrature_weights()'</code> — returns only weights, not (nodes, weights) tuple	Use <code>'LaguerreQuadrature(n).integrate(f)'</code>

'recommend_usage()' Legendre snippet (lines 789-800)	Generates code with non-existent 'integrate_transformed()'	Generate correct usage pattern
'recommend_usage()' Chebyshev snippet (lines 801-810)	Generates code with non-existent 'gauss_chebyshev_quadrature()'	Generate correct usage pattern

Implementation Order

1. **Fix `_integrate_legendre()`** — replace `integrate_transformed` call
2. **Fix `_integrate_chebyshev()`** — replace both singularity and non-singularity paths
3. **Fix `_integrate_laguerre()`** — use class-based API
4. **Fix `recommend_usage()` Legendre snippet** — generate correct code
5. **Fix `recommend_usage()` Chebyshev snippet** — generate correct code

Verification

After changes, run:

```
python api/quadrature_analyzer.py
```

All 7 demo examples should execute without `AttributeError` or unpacking errors.

Files Modified (Only This One)

- `api/quadrature_analyzer.py` — the only file that needs modification

Files NOT Modified (Read-Only Constraint)

- `chebyshev/*` — never modified
- `hermite/*` — never modified
- `laguerre/*` — never modified
- `legendre/*` — never modified